

# Enterprise DevOps Platform

## Final Project Report

Design and Development of a Complete Enterprise DevOps Ecosystem with Cloud Infrastructure, Automation, Security, Monitoring, Reliability and Self-Healing Capabilities

**GitHub Repository:** [github.com/karthi0079/enterprise-devops](https://github.com/karthi0079/enterprise-devops)

**Date:** September 2026

# 1. Executive Summary

This project implements a practical DevOps ecosystem covering cloud infrastructure provisioning, Infrastructure as Code, containerization, CI/CD automation, security scanning, container registry management, monitoring, centralized application logging, and Kubernetes self-healing.

AWS infrastructure is provisioned using Terraform. The application is containerized with Docker and published to GitHub Container Registry (GHCR). GitHub Actions automates image building, application testing, Trivy vulnerability scanning, image publishing, and deployment to an AWS EC2 instance.

CloudWatch provides EC2 monitoring and application log collection. Kubernetes/Minikube demonstrates orchestration, service exposure, two-replica deployment, and automatic pod recovery.

# 2. Project Objectives

- Cloud infrastructure provisioning using Terraform
- AWS networking and compute configuration
- Infrastructure as Code validation and version control
- Docker-based application containerization
- Automated CI/CD using GitHub Actions
- Container vulnerability scanning using Trivy
- Container image management using GHCR
- Automated deployment to AWS EC2
- CloudWatch monitoring and centralized application logging
- Kubernetes deployment and self-healing using multiple replicas
- Cost-conscious cloud architecture
- Technical documentation and operational evidence

# 3. Technology Stack

| Category               | Technology                                        |
|------------------------|---------------------------------------------------|
| Cloud                  | AWS                                               |
| Infrastructure as Code | Terraform                                         |
| Compute                | Amazon EC2                                        |
| Networking             | VPC, Public Subnet, Internet Gateway, Route Table |
| Security               | IAM Role, Security Group, Trivy                   |
| Containerization       | Docker                                            |
| Registry               | GitHub Container Registry                         |
| CI/CD                  | GitHub Actions                                    |
| Monitoring             | Amazon CloudWatch                                 |
| Logging                | CloudWatch Logs                                   |
| Orchestration          | Kubernetes / Minikube                             |
| Source Control         | Git / GitHub                                      |

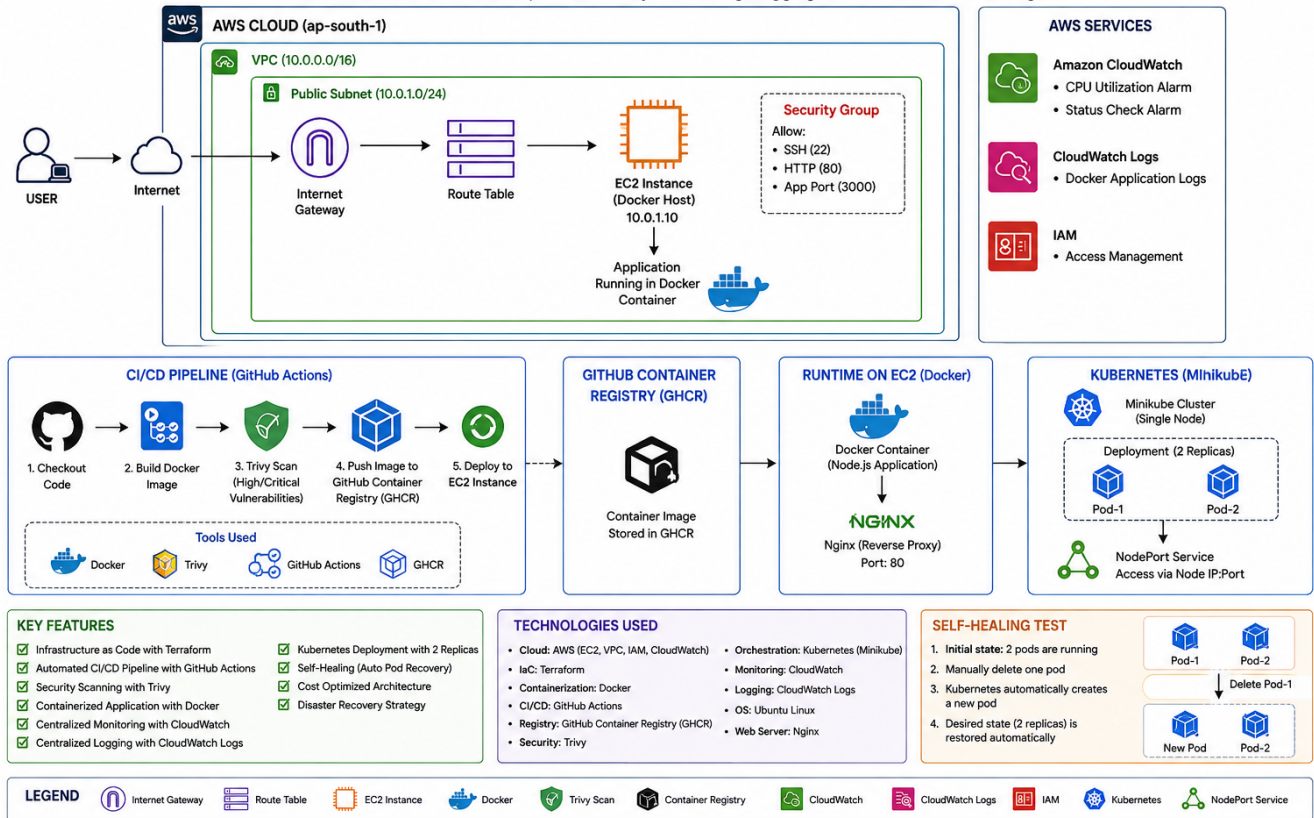
# 4. Architecture Diagram

The following diagram represents the implemented DevOps architecture, including the AWS VPC, public subnet, EC2/Docker deployment, GitHub Actions CI/CD pipeline, GHCR, CloudWatch monitoring and logging, and the local Kubernetes/Minikube

deployment with two replicas and self-healing.

## ENTERPRISE DEVOPS ECOSYSTEM – ARCHITECTURE DIAGRAM

Cloud Infrastructure, CI/CD Pipeline, Security, Monitoring, Logging & Kubernetes (Self-Healing)



## 5. AWS Infrastructure

### 5.1 VPC and Networking

The project uses an AWS VPC with CIDR block 10.0.0.0/16. The EC2 instance is deployed in a public subnet using the 10.0.1.0/24 address range. Public IP assignment is enabled and the subnet uses an Internet Gateway for internet connectivity.

### 5.2 Route Table

The public route table contains 10.0.0.0/16 → local and 0.0.0.0/0 → Internet Gateway.

### 5.3 EC2

The application is hosted on an EC2 instance running Docker. The instance uses an IAM role and a Security Group. The deployed application was successfully accessed through HTTP.

### 5.4 Security Group

| Protocol | Port | Source    | Purpose |
|----------|------|-----------|---------|
| TCP      | 22   | 0.0.0.0/0 | SSH     |
| TCP      | 80   | 0.0.0.0/0 | HTTP    |

For production, SSH should be restricted to trusted source IPs or replaced with a managed access mechanism.

## 6. Infrastructure as Code

Terraform manages the AWS infrastructure. The configuration is organized into vpc.tf for networking, ec2.tf for compute, iam.tf for IAM, monitoring.tf for CloudWatch resources, variables.tf for variables, outputs.tf for outputs, and provider.tf for provider configuration.

Terraform was used to validate, plan, apply, and manage infrastructure changes.

## 7. Docker Containerization

The application is packaged as a Docker image and exposes port 80. The container is deployed to EC2 using the GHCR image, port mapping 80:80, and the restart policy unless-stopped.

The final EC2 container was verified to use the Docker awslogs logging driver so application logs can be sent to CloudWatch Logs.

## 8. CI/CD Pipeline

GitHub Actions provides the automated CI/CD pipeline.

- Checkout source code
- Build Docker image
- Run the application container for testing
- Test the HTTP endpoint
- Run Trivy security scanning
- Authenticate with GHCR
- Push SHA-tagged and latest images
- Connect to EC2 using SSH
- Pull the latest image
- Replace the previous application container
- Start the new container with automatic restart and CloudWatch logging

GitHub Actions Secrets are used for deployment credentials.

## 9. DevSecOps and Trivy

Trivy is integrated into the GitHub Actions pipeline to scan the Docker image for HIGH and CRITICAL vulnerabilities. The scan acts as a security gate and can stop the workflow when matching vulnerabilities are detected.

During development, vulnerabilities were identified in Alpine/OpenSSL packages. The scan output identified fixed package versions, demonstrating vulnerability detection and remediation as part of the development lifecycle.

## 10. GitHub Container Registry

GitHub Container Registry stores the application image: ghcr.io/karthi0079/enterprise-devops/enterprise-devops-app. The pipeline publishes both a commit SHA image tag and the latest image tag. The EC2 instance successfully pulled the image from GHCR before deployment.

## 11. Monitoring and Logging

### CloudWatch Alarms

A CPU utilization alarm was configured with an 80% threshold using two evaluation periods of five minutes. An EC2 status-check alarm was also configured to detect failed instance status checks. Both alarms reached the OK state after CloudWatch received sufficient metric data.

### Application Logs

A CloudWatch log group named enterprise-devops-application was created with seven-day retention. The EC2 Docker container was verified to use the awslogs logging driver.

## 12. Kubernetes Deployment

Kubernetes was demonstrated locally using Minikube to avoid the additional cost of Amazon EKS.

The application uses a Kubernetes Deployment with two replicas and a NodePort Service. The service exposes port 80 through NodePort 30080.

The deployment successfully reported 2/2 replicas available and both application pods reached the Running state.

## 13. Self-Healing Demonstration

Both application pods were deliberately deleted using `kubectl delete pod -l app=enterprise-devops-app`. Kubernetes automatically created replacement pods, restoring the desired replica count.

**Result:** Desired replicas: 2 | Available replicas: 2 | Self-healing: Successful.

## 14. Reliability

- Docker restart policy: unless-stopped
- Kubernetes Deployment with two replicas
- Automatic Kubernetes pod recreation
- EC2 status monitoring
- CloudWatch alarms
- Infrastructure recreation through Terraform
- Version-controlled application and infrastructure configuration

## 15. Disaster Recovery Strategy

The application is stateless and does not use a database. Recovery can be performed by recreating the AWS infrastructure with Terraform, pulling the application image from GHCR, starting the Docker container, and verifying the HTTP endpoint.

The current project does not implement automated database or EBS snapshot backups because the application is stateless and does not use persistent database data. Terraform configuration and application source code are maintained in GitHub.

Future production improvements include EBS snapshots, S3 backup storage, cross-region recovery, database point-in-time recovery where applicable, automated disaster recovery testing, and multi-AZ architecture.

## 16. Cost Optimization

- Single EC2 instance
- Public subnet without a NAT Gateway
- No Application Load Balancer
- No RDS
- No EKS
- Local Minikube for Kubernetes demonstration
- Seven-day CloudWatch log retention
- No unnecessary production-scale managed services

## 17. Testing and Validation

- Terraform validation, plan, and apply
- AWS VPC, route table, Internet Gateway, Security Group, and EC2 verification
- Docker image build and HTTP application test
- GitHub Actions workflow execution
- Trivy security scan
- GHCR image push and EC2 deployment
- CloudWatch CPU and status-check alarms
- CloudWatch log group and awslogs configuration
- Minikube node, Deployment, Pod, and Service verification
- Kubernetes pod deletion and automatic replacement

## 18. Evidence and Screenshots

The final submission includes captured evidence for AWS infrastructure, Terraform apply, GitHub Actions, Trivy scan, GHCR, EC2 deployment, CloudWatch alarms/logging, Kubernetes nodes/pods/deployment/service, self-healing, and successful application responses.

Project screenshots should be inserted after the relevant sections or collected in an appendix.

## 19. Repository Structure

```
enterprise-devops/  
■■■■ .github/workflows/ci-cd.yml  
■■■■ app/  
■■■■ terraform/  
■■■■ kubernetes/  
■■■■ monitoring/  
■■■■ security/  
■■■■ docs/  
■■■■ README.md
```

## 20. Conclusion

The Enterprise DevOps Platform demonstrates an end-to-end DevOps workflow. Infrastructure is managed using Terraform, the application is containerized using Docker, and GitHub Actions automates build, test, security scanning, image publishing,

and EC2 deployment.

Trivy introduces security scanning into the CI/CD lifecycle. GHCR provides container image storage. CloudWatch provides infrastructure monitoring and centralized application logging.

Kubernetes/Minikube demonstrates orchestration using two replicas and successfully proves self-healing through automatic pod recreation after deliberate pod deletion.

The resulting architecture provides a practical, reproducible, version-controlled, and cost-conscious DevOps implementation suitable for demonstrating cloud infrastructure, automation, DevSecOps, observability, and reliability concepts.

## 21. Final Status

| Capability              | Status              |
|-------------------------|---------------------|
| AWS Infrastructure      | Completed           |
| Terraform IaC           | Completed           |
| Docker Containerization | Completed           |
| GitHub Actions CI/CD    | Completed           |
| Trivy Security Scanning | Completed           |
| GHCR Registry           | Completed           |
| EC2 Deployment          | Completed           |
| CloudWatch Monitoring   | Completed           |
| CloudWatch Logging      | Completed           |
| Kubernetes Deployment   | Completed           |
| Two-Replica Deployment  | Completed           |
| Kubernetes Self-Healing | Tested Successfully |
| Documentation           | Completed           |
| Screenshots/Evidence    | Captured            |